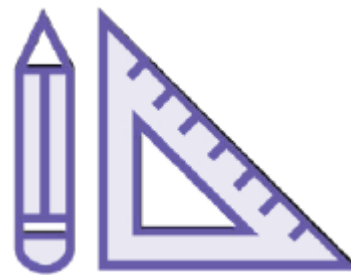




集合迭代器

北京理工大学计算机学院
金旭亮

数据集合的三种主要操作



1. 遍历集合中的所有元素
2. 按照内容在集合中查找对象
3. 向集合中插入、删除和修改对象



JDK中的各个集合类，提供了相应的方法（这些方法通常由这些类所实现的接口而确定）实现上述操作。

集合的遍历操作

- 最常用的一种集合操作是遍历操作，也就是一个一个地逐个访问集合中的所有元素。
- 诸如数组、ArrayList等对象集合，支持传统的for循环。
- JDK为各种集合遍历提供了另一种“通用”的方式——迭代器对象（Iterator），为了简化其代码编写，JDK 5（及以上版本）提供了foreach循环来遍历集合。



Iterator可以适用于所有的集合。
Java的foreach循环，其实就是通过Iterator实现的。

使用迭代器遍历集合



Iterator<E>接口定义了3个方法：

```
public Iterator<E> {  
    boolean hasNext(); // 是否还有下一个元素  
    E next();          // 获取下一个元素  
    void remove();     // 移除iterator返回的上一个元素  
}
```



实现了Iterator<E>接口的对象称为“**迭代器**”，使用它可以遍历集合。

使用迭代器遍历集合

- 所有实现了Collection接口的集合对象，都提供了一个iterator()方法，以获取一个iterator对象：

在实际开发中，iterator用起来比较麻烦，大多数场景下，会用for或forEach循环取代它。

```
public class TestIterator {  
    public static void main(String[] args) {  
        //创建一个集合  
        Collection<String> books = new HashSet<>();  
        books.add("One book");  
        books.add("Two book");  
        books.add("Three book");  
        //获取books集合对应的迭代器  
        Iterator<String> it = books.iterator();  
        //使用迭代器遍历集合  
        while (it.hasNext()) {  
            String book = (String) it.next();  
            System.out.println(book);  
        }  
        System.out.println("集合中的元素为: " + books);  
        //在迭代的过程中同时加工每个元素  
        it.forEachRemaining(book->{  
            System.out.println(book.toUpperCase());  
        } );  
    }  
}
```

示例 : TestIterator.java

集合的ForEach遍历

- JDK 5以后，Collection接口派生自Iterable，因此JDK中内置的**所有**集合都可以使用新的foreach语句完成遍历工作。

```
public class TestForeach {  
    public static void main(String[] args) {  
        //创建一个集合  
        Collection<String> books = new HashSet<String>();  
        books.add("One book");  
        books.add("Two book");  
        books.add("Three book");  
        for (var book : books) {  
            System.out.println(book);  
        }  
        System.out.println(books);  
        //使用Lambda表达式特性 (方法引用) 可以方便地遍历集合  
        books.forEach(System.out::println);  
    }  
}
```

示例: TestForeach.java

foreach循环

集合对象有一个forEach()方法，它可以接收一个Lambda表达式，可用于遍历集合元素，它与foreach循环不是一回事，foreach循环是一种语法特性，forEach则是JDK为集合类型提供的一个方法。

动手动脑



- 有这么一种说法：

Java的foreach循环是一个“语法糖”，它实际上基于Iterator实现的，是对使用Iterator遍历集合代码的一种简化，或者说，它是使用Iterator的代码的一个简化封装。

- 请编写一个使用foreach的示例程序，然后使用javap（或jclasslib之类的反汇编工具）验证上述说法，看看这种说法是否正确。

遍历集合时不要修改集合！

- 当使用for each或迭代器循环遍历集合时，注意不要再向集合中追加或删除元素，否则，会引发`ConcurrentModificationException`。
- 请参看以下示例：

//创建一个集合

```
Collection<String> books = new HashSet<>();  
books.add("One book");  
books.add("Two book");  
books.add("Three book");  
System.out.println("集合中包容的所有元素: "+books);
```

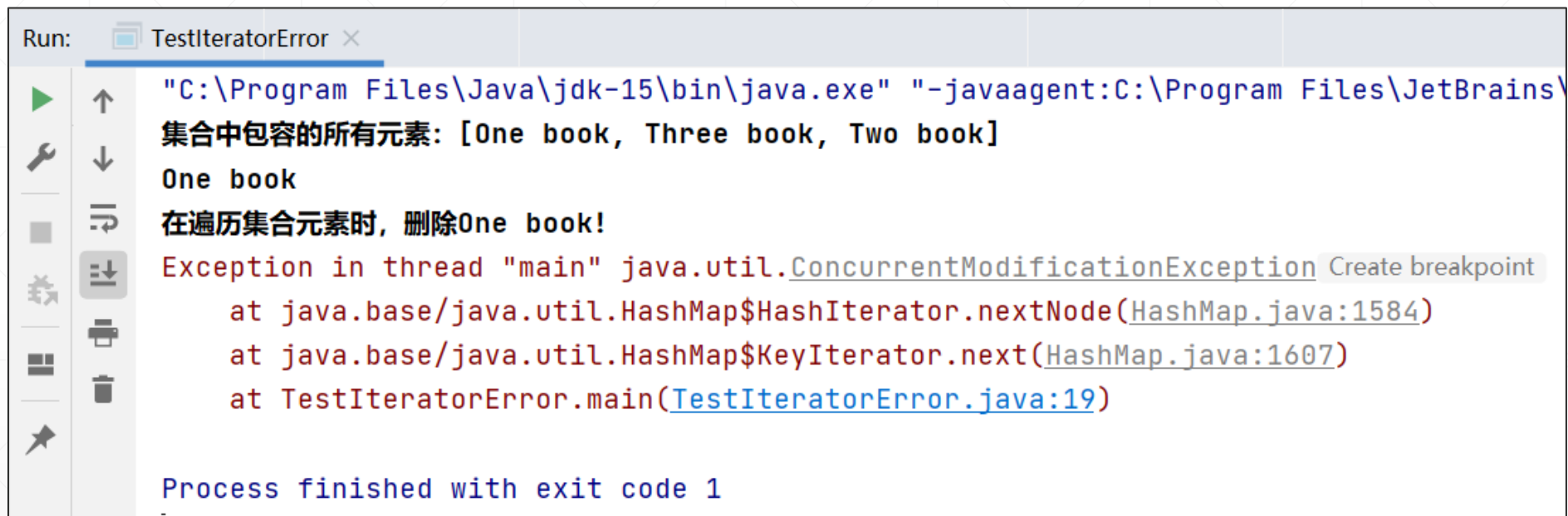
TestIteratorError.java

//获取books集合对应的迭代器

```
Iterator<String> it = books.iterator();  
//使用Iterator迭代过程中，不可修改集合元素！  
while(it.hasNext()) {  
    String book = it.next();  
    System.out.println(book);  
    if (book.equals("One book")) {  
        System.out.println("在遍历集合元素时，删除One book! ");  
        books.remove(book);   
    }  
}
```

示例程序运行截图

由于在循环遍历集合过程中，尝试删除第一个元素，结果导致异常被抛出：



```
Run: TestIteratorError x
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\
集合中包容的所有元素: [One book, Three book, Two book]
One book
在遍历集合元素时，删除One book!
Exception in thread "main" java.util.ConcurrentModificationException Create breakpoint
    at java.base/java.util.HashMap$HashIterator.nextNode(HashMap.java:1584)
    at java.base/java.util.HashMap$KeyIterator.next(HashMap.java:1607)
    at TestIteratorError.main(TestIteratorError.java:19)

Process finished with exit code 1
```



这里有一个很灵异的现象：上述异常并不必然会抛出，比如，本例中如果将删除的元素改为“Two book”，则程序可以顺利运行，不会有任何的异常抛出，试一试！

百思不得其解!

```
//灵异现象——如果删除"Two book",本示例不会引发异常,为什么呢?  
while(it.hasNext()) {  
    String book = it.next();  
    System.out.println(book);  
    if (book.equals("Two book")) {  
        System.out.println("在遍历集合元素时, 删除Two book! ");  
        //这句现在不引发异常了!  
        books.remove(book);  
    }  
}
```

“One book” 和 “Two book”，
这两个元素有什么不一样？



```
TestIteratorError x  
↑  
↓  
↺  
↻  
⇅  
☐  
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaa  
数组中包容的所有元素: [One book, Three book, Two book]  
One book  
Three book  
Two book  
在遍历集合元素时, 删除Two book!  
移除元素之后: [One book, Three book]
```

探索技术的底层实现细节.....

关键在于弄清楚，示例在哪个地方抛出的 **ConcurrentModificationException**

查询HashSet的源码，发现它内部封装的是HashMap，HashMap里面定义了一个HashIterator用于实现集合的遍历，示例代码中调用了它的hasNext方法和nextNode方法。

```
abstract class HashIterator {
    Node<K,V> next;           // next entry to return
    Node<K,V> current;        // current entry
    int expectedModCount;     // for fast-fail
    int index;                 // current slot

    HashIterator() {...}

    public final boolean hasNext() {
        return next != null;
    }
}
```

```
final Node<K,V> nextNode() {
    Node<K,V>[] t;
    Node<K,V> e = next;
    if (modCount != expectedModCount)
        throw new ConcurrentModificationException();
    if (e == null)
        throw new NoSuchElementException();
    if ((next = (current = e).next) == null && (t = table) != null) {
        do {} while (index < t.length && (next = t[index++]) == null);
    }
    return e;
}
```

异常是在这个方法中抛出来的.....

"Two book"是最后一个元素.....



集合中包容的所有元素: [One book, Three book, Two book]

```
//灵异现象——如果删除"Two book",本示例不会引发异常,为什么呢?  
while(it.hasNext()) {  
    String book = it.next();  
    System.out.println(book);  
    if (book.equals("Two book")) {  
        System.out.println("在遍历集合元素时, 删除Two book! ");  
        //这句现在不引发异常了!  
        books.remove(book);  
    }  
}
```

对比前面的源代码, 发现 hasNext 方法是不会抛出异常的, 异常是由 it.next() 这句引发的。

再看看集合中元素的排列顺序, 发现 Two book 是最后一个元素, 由此真相大白!



在遍历过程中删除集合的最后一个元素后, hasNext 返回 false, 导致 it.next() 这句根本没执行, 所以, 就不会引发异常了。

遍历时删除元素的正确方法

- 如果在遍历时非要删除集合中的元素，则必须通过迭代器对象的remove方法，而不能通过集合对象直接删除。
- 典型代码如下：

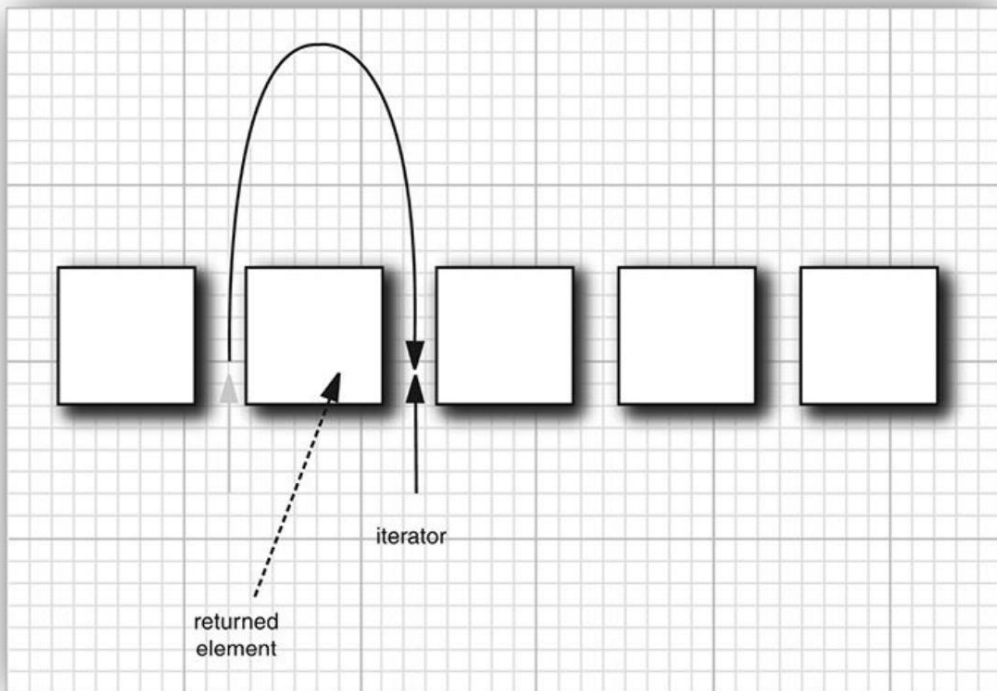
```
//获取books集合对应的迭代器
Iterator<String> it = books.iterator();
while(it.hasNext()) {
    String book = it.next();
    System.out.println(book);
    if (book.equals("One book")) {
        //使用Iterator迭代过程中，通过它来移除集合元素是安全的
        it.remove();
    }
}
```

参看示例：TestIteratorSucceed.java

尽管确实可以在迭代时移除当前访问的元素，但并不推荐这么做，特别是在多线程环境下，会带来很大的麻烦。

推荐的方式是使用Stream API中的filter()方法，过滤掉不需要的元素，而不是直接修改原始的数据集合。

理解Iterator



迭代器类似于一个书签，它“夹”在两页书中间，起到一个定位的作用。

当调用`next`方法时，迭代器就越过下一个元素，并返回刚刚越过的那个元素的引用给外界。

`remove`方法删除上次调用`next`方法时返回的元素。如果调用`remove`之前没有调用`next`将是不合法的，会抛出一个`IllegalStateException`异常。

不允许删除的集合

```
public class UseUnmodifiedCollection {  
    public static void main(String[] args) {  
        var books = new ArrayList<String>();  
        books.add("One book");  
        books.add("Two book");  
        books.add("Three book");  
        var readonlyBooks : Collection<String> =  
            Collections.unmodifiableCollection(books);  
        var iterator : Iterator<String> = readonlyBooks.iterator();  
        while (iterator.hasNext()) {  
            System.out.println(iterator.next());  
            iterator.remove();  
        }  
    }  
}
```

引发

示例: UseUnmodifiedCollection.java

使用Iterator遍历集合时，虽然可以删除元素。但实际开发中不建议使用这一特性，为避免元素被删除，可以创建只读的集合，彻底禁用删除功能。



```
Run: UseUnmodifiedCollection x  
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA  
One book  
Exception in thread "main" java.lang.UnsupportedOperationException Create breakpoint  
    at java.base/java.util.Collections$UnmodifiableCollection$1.remove(Collections.java:1051)  
    at UseUnmodifiedCollection.main(UseUnmodifiedCollection.java:14)  
  
Process finished with exit code 1
```

“普通”Iterator的局限性



仅支持向前的导航。



仅支持删除，不支持添加和替换集合元素。

JDK中提供了另一个功能更强的Iterator——`ListIterator`，支持前后移动，JDK中的`ArrayList`、`LinkedList`等都使用它。

功能增强的ListIterator

ListIterator比普通的Iterator功能增强的一点在于，它支持反向遍历.....

```
//反向遍历
```

```
private static void visitListFromEndToBegin(ArrayList<String> books) {  
    ListIterator<String> iterator = books.listIterator(books.size());  
    while (iterator.hasPrevious()) {  
        System.out.println(iterator.previous());  
    }  
}
```

通过ListIterator修改集合

//遍历的同时修改集合

```
private static void modifyListWhenVisit(ArrayList<String> books) {  
    ListIterator<String> iterator;  
    iterator = books.listIterator();  
    int index = 0;  
    while (iterator.hasNext()) {  
        System.out.println(iterator.next());  
        //要先调用next()方法, 之后才能更改它的元素  
        index++;  
        iterator.set(String.valueOf(index));  
    }  
    //输出修改后的结果  
    books.forEach(System.out::println);  
}
```

ListIterator可以在遍历之后, 立即修改元素



ListIterator是有序的, 它的remove方法可以删除“左”或“右”的元素。

使用多个Iterator同时遍历集合

//同一个集合，返回的Iterator是独立的，互不影响。

```
private static void iteratorAreNotSame() {  
    var nums : List<Integer> = List.of(1, 2, 3, 4, 5);  
    Random random = new Random();  
    Runnable runnable = () -> {  
        var iterator : ListIterator<Integer> = nums.listIterator();  
        while (iterator.hasNext()) {  
            try {  
                //随机休眠若干时间  
                Thread.sleep(random.nextInt(bound: 2000));  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            System.out.println(Thread.currentThread().getName() +  
                               ":" + iterator.next());  
        }  
    };  
    new Thread(runnable).start();  
    new Thread(runnable).start();  
}
```

多次调用listIterator()方法，可以得到多个迭代器，它们彼此是相互独立的，可以以不同的步调去访问集合。



```
Run: TestListIterator x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
Thread-1:1  
Thread-0:1  
Thread-1:2  
Thread-1:3  
Thread-0:2  
Thread-1:4  
Thread-1:5  
Thread-0:3  
Thread-0:4  
Thread-0:5  
  
Process finished with exit code 0
```

自定义支持迭代的集合

支持迭代访问的类，必须实现Iterable<T>接口

```
//一个自定义的，支持foreach迭代的数字序列生成器类
public class NumberSequence implements Iterable<Integer> {
    final int start, increment, limit;

    public NumberSequence(int start, int increment, int limit) {
        this.start = start;
        this.increment = increment;
        this.limit = limit;
    }

    @Override
    public Iterator<Integer> iterator() {
        //返回一个自定义的迭代器给外界
        return new SequenceIterator(this);
    }
}
```

通常情况下，直接使用JDK中现成的集合类型及它们内置的迭代器就足够了，但如果需要，我们也可以定义迭代器，从而以特定的方式遍历特定的集合。

实现自定义迭代器

自定义迭代器必须实现`Iterator<E>`接口，为它所定义的`hasNext`和`next`方法提供实现代码：

```
class SequenceIterator implements Iterator<Integer> {  
    //数据源  
    private NumberSequence sequence;  
    //向外界返回下一个值  
    private int nextValue;  
  
    public SequenceIterator(NumberSequence sequence) {  
        this.sequence = sequence;  
        this.nextValue = sequence.start;  
    }  
}
```

迭代器的主要职责，就是一次提供一个数据给外界.....

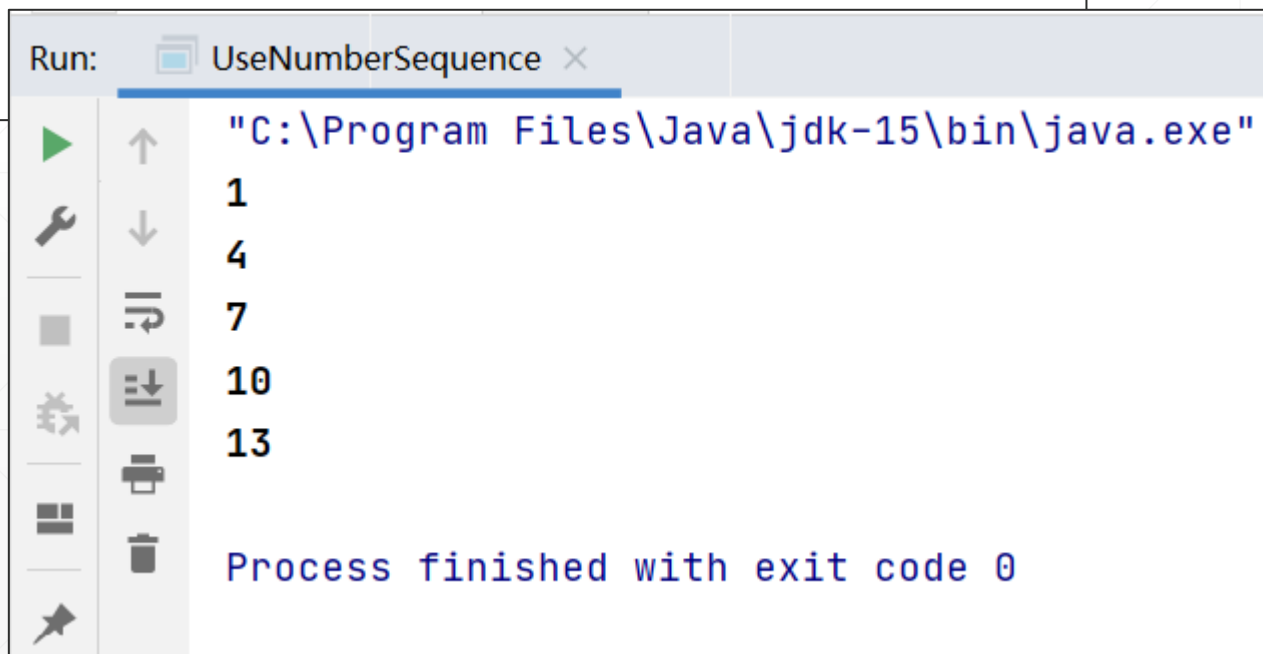
```
@Override  
public boolean hasNext() {  
    return this.nextValue <= this.sequence.limit;  
}  
  
@Override  
public Integer next() {  
    if (this.sequence.limit < this.nextValue) {  
        throw new NoSuchElementException("No such element");  
    }  
    int rv = this.nextValue;  
    //生成下一个值  
    this.nextValue += this.sequence.increment;  
    return rv;  
}
```

使用自定义迭代器

生成起始值为1，最大值不超过15，每次增加3的等差数列

```
public class UseNumberSequence {  
    public static void main(String[] args) {  
        var sequence = new NumberSequence( start: 1, increment: 3, limit: 15);  
        for (Integer i : sequence) {  
            System.out.println(i);  
        }  
    }  
}
```

运行结果:



```
Run: UseNumberSequence x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
1  
4  
7  
10  
13  
Process finished with exit code 0
```

向集合中插入元素的两种方式

直接调用集合本身的方法，
比如add()方法

使用listIterator自己的
add()方法

两种方式，功能一样，
选哪种比较好？



实践出真知!

编写测试代码，分别使用两种方式插入大量的元素到特定集合中，检测它们的表现，哪个表现好，就选哪个!

```
//将dataToAdd集合中的所有元素，使用List.add方法，  
//全部插入到List集合的中部，统计插入耗费的时间  
public static long insertMid(List<String> list,  
                             List<String> dataToAdd) {  
    int target = list.size() / 2;  
    long start = System.nanoTime();  
  
    for (String s : dataToAdd) {  
        list.add(target++, s);  
    }  
    long end = System.nanoTime();  
    return end - start;  
}
```

方式一

```
//将dataToAdd集合中的所有元素，使用ListIterator.add方法，  
//全部插入到List集合的中部，统计插入耗费的时间  
public static long insertIter(List<String> list,  
                              List<String> dataToAdd) {  
    int target = list.size() / 2;  
    long start = System.nanoTime();  
    ListIterator<String> iter = list.listIterator(target);  
    for (String s : dataToAdd) {  
        iter.add(s);  
    }  
    long end = System.nanoTime();  
    return end - start;  
}
```

方式二

实地测试

赛前准备:

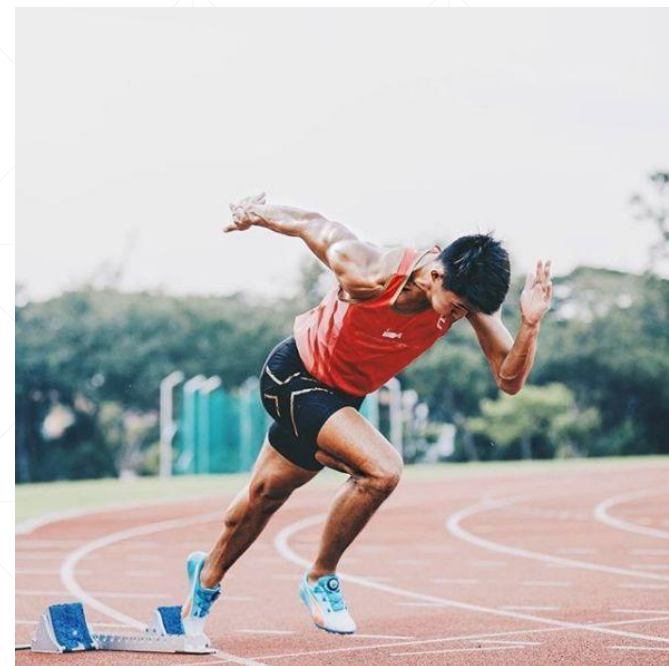
```
//生成25万个整数, 追加到dataToInsert这个集合中  
final int COUNT = 250_000;  
String [] items = new String[COUNT];  
for (int i = 0; i < COUNT; i++) {  
    items[i] = "" + i;  
}  
List<String> dataToInsert = Arrays.asList(items);
```



高手对决!

```
//分别以两种方式, 将25万个整数插入到LinkedList中
//方式一: 使用List.Add()方法
List<String> linkedList = new LinkedList<>();
linkedList.addAll(dataToInsert);
System.out.printf("使用List.Add向LinkedList中部插入250000个元素, 耗时: "
    + "%,12.6f secs\n",
    insertMid(linkedList, dataToInsert) / 1_000_000_000.0);
```

```
//方式二: 使用ListIterator.Add()方法
linkedList = new LinkedList<>();
linkedList.addAll(dataToInsert);
System.out.printf("使用ListIterator向LinkedList中部插入250000个元素, 耗时: "
    + "%,12.6f secs\n",
    insertIter(linkedList, dataToInsert) / 1_000_000_000.0);
```



实地测试两种数据
插入方式的耗时

比赛结果

```
Run: ListInsertComparisons x
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files
使用List.Add向LinkedList中部插入250000个元素, 耗时: 107.527179 secs
使用ListIterator向LinkedList中部插入250000个元素, 耗时: 0.011146 secs
Process finished with exit code 0
```

ListIterator 完胜!



小结



本讲重点介绍了集合的foreach遍历和迭代器的基本用法，JDK中的各种集合普遍适用。



通常情况下，对集合进行遍历时，不要修改集合，如果确实需要删除元素，必须使用iterator的remove()方法去删除，如果需要修改元素，可以使用ListIterator的set()方法，插入大量元素，推荐使用ListIterator的add()方法。



本讲示例还展示了删除集合元素时出现的“灵异”现象，通过查找其出现的原因，我们知道了阅读JDK源码的重要性。通过阅读源码，对于特定的现象，我们就能“知其所以然”，这是学好Java的一种重要方法。